

Script.sql

```
1  SELECT
2      b.brand_name,
3      COUNT(o.order_id) AS total_orders,
4      ROUND(SUM(o.gross_revenue), 2) AS total_gross_revenue,
5      ROUND(SUM(discount_applied), 2) AS total_discount,
6      ROUND(SUM(o.gross_revenue - o.discount_applied),
7          2) AS total_net_revenue,
8      ROUND(AVG(o.gross_revenue), 2) AS avg_order_value,
9      ROUND(AVG(o.gross_revenue - o.discount_applied),
10         2) AS avg_net_revenue
11 FROM
12     fact_orders o
13     JOIN
14     dim_brand b ON o.brand_id = b.brand_id
15 WHERE
16     o.order_status = 'Delivered'
17 GROUP BY b.brand_name
18 ORDER BY total_gross_revenue DESC;
```

brand_name	total_orders	total_gross_revenue	total_discount	total_net_revenue	avg_order_value	avg_net_revenue
Biryani Brand	12698	3552888.47	495768.71	3057119.76	279.8	240.76
Pizza Brand	7680	2687484.48	377209.1	2310275.38	349.93	300.82
Burger Brand	8790	1934122.26	273870.62	1660251.64	220.04	188.88
Chinese Brand	6360	1653739.83	229700.49	1424039.34	260.02	223.91
Healthy Bowl Brand	5212	1200581.51	158209.38	1042372.13	230.35	199.99
Dessert Brand	4206	757459.72	106789.52	650670.2	180.09	154.7

Brand Metrics Summary

This query analyzes the performance of each brand by calculating total orders, gross revenue, discounts, net revenue, and average order values using only delivered orders.

Script.sql

```
1  SELECT
2      b.brand_name,
3      SUM(CASE
4          WHEN o.order_status != 'Delivered' THEN 1
5          ELSE 0
6      END) AS canceled_refunded_order,
7      COUNT(o.order_id) AS total_order,
8      ROUND(SUM(CASE
9          WHEN o.order_status != 'Delivered' THEN 1
10         ELSE 0
11     END) * 100.0 / COUNT(o.order_id),
12     2) AS cancel_refund_pct
13 FROM
14     fact_orders o
15     JOIN
16     dim_brand b ON o.brand_id = b.brand_id
17 GROUP BY b.brand_name;
```

brand_name	canceled_refunded_order	total_order	cancel_refund_pct
Burger Brand	918	9708	9.46
Pizza Brand	927	8607	10.77
Biryani Brand	1451	14149	10.26
Chinese Brand	718	7078	10.14
Healthy Bowl Brand	572	5784	9.89
Dessert Brand	468	4674	10.01

Which brand has the highest number of cancelled or returned orders?

If a brand has massive total orders but a high cancel or refund pct, it means marketing is doing a great job bringing people in, but the kitchen operations or delivery fleet is failing them.

Script.sql

```
1  SELECT
2      b.brand_name,
3      ROUND(AVG(CASE WHEN o.order_status = 'Delivered' THEN o.food_cost END), 2) AS avg_delivered_food_cost,
4      ROUND(AVG(CASE WHEN o.order_status = 'Delivered' THEN o.packing_cost END), 2) AS avg_delivered_packing_cost
5  FROM fact_orders o
6  JOIN dim_brand b ON o.brand_id = b.brand_id
7  GROUP BY b.brand_name
8  ORDER BY avg_delivered_food_cost DESC;
```

brand_name	avg_delivered_food_cost	avg_delivered_packing_cost
Pizza Brand	120.33	21.99
Biryani Brand	115.56	20.01
Chinese Brand	96.28	18.01
Burger Brand	87.34	18.01
Healthy Bowl Brand	76	14.98
Dessert Brand	54.15	12.02

What is the average packing cost vs. average food cost for each brand?

This query highlights if a brand is overspending on premium packaging for low-ticket menu items.

```

1  WITH unit_econ AS (
2      SELECT
3          b.brand_name,
4          t.slot_name,
5          ROUND(SUM(o.gross_revenue),2) AS gross_revenue,
6          ROUND(SUM(o.gross_revenue - o.discount_applied),2) AS net_revenue,
7          ROUND(SUM((o.gross_revenue - o.discount_applied) * o.platform_commission_pct),2) AS commission_paid,
8          ROUND(SUM(o.food_cost + o.packing_cost + o.delivery_cost_brand),2) AS variable_cost,
9          ROUND(SUM(o.gross_revenue - o.discount_applied)
10             - SUM((o.gross_revenue - o.discount_applied) * o.platform_commission_pct)
11             - SUM(o.food_cost + o.packing_cost + o.delivery_cost_brand),2) AS contribution_margin
12     FROM fact_orders o
13     JOIN dim_brand b ON o.brand_id = b.brand_id
14     JOIN dim_time_slot t ON o.time_slot_id = t.time_slot_id
15     WHERE o.order_status = 'Delivered'
16     GROUP BY b.brand_name, t.slot_name
17 )
18 SELECT *,
19     ROUND(contribution_margin / NULLIF(net_revenue, 0) * 100.0, 2) AS cm_pct,
20     DENSE_RANK() OVER (PARTITION BY brand_name ORDER BY
21         contribution_margin / NULLIF(net_revenue, 0) DESC) AS slot_rank
22 FROM unit_econ;

```

Unit Economics per Brand per Time Slot

- This query evaluates the unit economics of each brand across different time slots by calculating gross revenue, net revenue, commission paid, variable costs, contribution margin, and contribution margin percentage for delivered orders.
- It also ranks the time slots within each brand based on contribution margin percentage, helping identify the most profitable time periods for every brand.

brand_name	slot_name	gross_revenue	net_revenue	commission_paid	variable_cost	contribution_margin	cm_pct	slot_rank
Biryani Brand	Lunch	1037143.42	892322.74	193906.14	661303.78	37112.82	4.16	1
Biryani Brand	Dinner	1401498.29	1206155.39	262449.74	894821.55	48884.09	4.05	2
Biryani Brand	Evening Snack	206740.74	178269.82	38566.81	132518.19	7184.82	4.03	3
Biryani Brand	Breakfast	151406.73	130669.41	28362.55	97060.47	5246.39	4.02	4
Biryani Brand	Brunch	452023.77	388800.37	85672.27	288241.22	14886.88	3.83	5
Biryani Brand	Late Night	304075.52	260902.03	57776.55	193884.71	9240.77	3.54	6
Burger Brand	Late Night	167822.73	143992.89	31142.96	110450.5	2399.43	1.67	1
Burger Brand	Lunch	426578.76	366167.42	79646.01	281065.14	5456.27	1.49	2
Burger Brand	Dinner	712740.8	611758.28	132120.09	470551.26	9086.93	1.49	3
Burger Brand	Brunch	229242.57	197447.83	42487.27	152144.95	2815.61	1.43	4
Burger Brand	Evening Snack	292679.63	251156.99	54564.32	193926.41	2666.26	1.06	5
Burger Brand	Breakfast	105057.77	89728.23	19494.21	89342.14	-19108.12	-21.3	6
Chinese Brand	Brunch	146259.8	125531.09	26287.53	87276.03	11967.53	9.53	1
Chinese Brand	Lunch	366688.9	316520.93	66539.97	220489.88	29491.08	9.32	2
Chinese Brand	Late Night	222759.7	191664.62	40669.51	133226.24	17768.87	9.27	3

```

Script.sql
1  SELECT
2      b.brand_name,
3      p.platform_name,
4      t.slot_name,
5      SUM(CASE
6          WHEN o.order_status != 'Delivered' THEN 1
7          ELSE 0
8      END) AS canceled_refunded_order,
9      COUNT(o.order_id) AS total_order,
10     ROUND(SUM(CASE
11         WHEN o.order_status != 'Delivered' THEN 1
12         ELSE 0
13     END) * 100.0 / COUNT(o.order_id),
14     2) AS cancel_refund_pct
15 FROM
16     fact_orders o
17     JOIN
18     dim_brand b ON o.brand_id = b.brand_id
19     JOIN
20     dim_platform p ON o.platform_id = p.platform_id
21     JOIN
22     dim_time_slot t ON o.time_slot_id = t.time_slot_id
23 GROUP BY b.brand_name, p.platform_id, t.slot_name;

```

brand_name	platform_name	slot_name	canceled_refunded_order	total_order	cancel_refund_pct
Burger Brand	Direct	Dinner	22	364	6.04
Burger Brand	Direct	Brunch	14	133	10.53
Burger Brand	Direct	Lunch	22	212	10.38
Burger Brand	Direct	Late Night	4	84	4.76
Burger Brand	Direct	Evening Snack	14	147	9.52
Burger Brand	Direct	Breakfast	3	50	6.00
Burger Brand	Swiggy	Breakfast	23	198	11.62
Burger Brand	Swiggy	Dinner	109	1274	8.56
Burger Brand	Swiggy	Brunch	48	403	11.91
Burger Brand	Swiggy	Lunch	76	766	9.92
Burger Brand	Swiggy	Evening Snack	45	523	8.60
Burger Brand	Swiggy	Late Night	29	299	9.70
Burger Brand	Zomato	Brunch	65	634	10.25
Burger Brand	Zomato	Lunch	130	1192	10.91
Burger Brand	Zomato	Late Night	41	451	9.09
Burger Brand	Zomato	Dinner	176	1900	9.26
Burger Brand	Zomato	Evening Snack	74	799	9.26
Burger Brand	Zomato	Breakfast	23	279	8.24
Pizza Brand	Direct	Late Night	14	117	11.97
Pizza Brand	Direct	Evening Snack	11	105	10.48
Pizza Brand	Direct	Dinner	23	354	6.50
Pizza Brand	Direct	Lunch	20	228	8.77
Pizza Brand	Direct	Brunch	13	169	7.69
Pizza Brand	Direct	Breakfast	8	95	8.42
Pizza Brand	Swiggy	Lunch	89	783	11.37

Multi-dimensional Grain Analysis (Brand + Platform + Time Slot)

- This query analyzes order performance at the combined level of brand, platform, and time slot by calculating total orders, cancelled / refunded orders, and the cancellation/refund percentage.
- It helps identify which brand-platform-time slot combinations experience the highest cancellation rates, enabling better operational and customer service decisions.

```

Script.sql
1  WITH contribution_margin_cte AS (
2      SELECT
3          t.day_type,
4          (o.gross_revenue - o.discount_applied) AS net_revenue,
5          (o.gross_revenue - o.discount_applied) -
6          (((o.gross_revenue - o.discount_applied) * o.platform_commission_pct) + o.food_cost + o.packing_cost + o.delivery_cost_brand)
7          AS contribution_margin
8      FROM fact_orders o
9      JOIN dim_time_slot t
10     ON o.time_slot_id = t.time_slot_id
11     WHERE o.order_status = 'Delivered'
12 )
13 SELECT
14     day_type,
15     COUNT(*) AS total_orders,
16     ROUND(SUM(contribution_margin), 2) AS total_margin,
17     ROUND(AVG(contribution_margin), 2) AS avg_margin_per_order,
18     ROUND(SUM(contribution_margin) * 100.0 / NULLIF(SUM(net_revenue), 0), 2) AS Weighted_cm_pct
19 FROM contribution_margin_cte
20 GROUP BY day_type;

```

day_type	total_orders	total_margin	avg_margin_per_order	Weighted_cm_pct
Weekend	24942	480763.25	19.28	8.42
Weekday	20004	396745.16	19.83	8.94

How does the Contribution Margin % (CM%) behave on Weekends vs. Weekdays?

Operational Recommendation:

The company should allocate 60% of performance marketing budget exclusively to Friday-Sunday campaigns, as physical kitchen spaces yield disproportionately higher cash velocity during weekend spikes.

```

1  SELECT
2      b.brand_name,
3      o.orders_date,
4      ROUND(SUM((o.gross_revenue - o.discount_applied) -
5          ((o.gross_revenue - o.discount_applied) * o.platform_commission_pct) -
6          (o.food_cost + o.packing_cost + o.delivery_cost_brand)),2) AS cm_by_date,
7      ROUND(SUM(
8          SUM((o.gross_revenue - o.discount_applied) -
9              ((o.gross_revenue - o.discount_applied) * o.platform_commission_pct) -
10             (o.food_cost + o.packing_cost + o.delivery_cost_brand))
11      ) OVER (
12          PARTITION BY b.brand_name
13          ORDER BY o.orders_date
14          RANGE BETWEEN INTERVAL 29 DAY PRECEDING AND CURRENT ROW
15      ),2) AS rolling_30d_contribution_margin
16
17 FROM fact_orders o
18 JOIN dim_brand b ON o.brand_id = b.brand_id
19 WHERE o.order_status = 'Delivered'
20 GROUP BY b.brand_name, o.orders_date;

```

brand_name	orders_date	cm_by_date	rolling_30d_contribution_margin
Biryani Brand	2023-01-01	172.82	172.82
Biryani Brand	2023-01-02	-5.05	167.77
Biryani Brand	2023-01-03	81.27	249.04
Biryani Brand	2023-01-04	190.33	439.36
Biryani Brand	2023-01-05	22.43	461.8
Biryani Brand	2023-01-06	208.28	670.08
Biryani Brand	2023-01-07	307.95	978.03
Biryani Brand	2023-01-08	355.24	1333.27
Biryani Brand	2023-01-09	83.54	1416.81
Biryani Brand	2023-01-10	15.17	1431.99
Biryani Brand	2023-01-11	127.79	1559.77
Biryani Brand	2023-01-12	164.22	1723.99
Biryani Brand	2023-01-13	117.3	1841.29
Biryani Brand	2023-01-14	665.01	2506.3
Biryani Brand	2023-01-15	128.79	2635.08
Biryani Brand	2023-01-16	63.41	2698.49
Biryani Brand	2023-01-17	-46.79	2651.7
Biryani Brand	2023-01-18	106.5	2758.21
Biryani Brand	2023-01-19	114.64	2872.84
Biryani Brand	2023-01-20	61.05	2933.89
Biryani Brand	2023-01-21	373.17	3307.06
Biryani Brand	2023-01-22	130.97	3438.03
Biryani Brand	2023-01-23	39	3477.03
Biryani Brand	2023-01-24	115.63	3592.66
Biryani Brand	2023-01-25	52.15	3644.81

Rolling 30-Day Contribution Margin Trend per Brand

It helps smooth out daily fluctuations, making it easier to identify long-term performance patterns and monitor whether a brand's profitability is improving or declining.

Script.sql

```
1  SELECT
2      brand_name,
3      DATE_FORMAT(orders_date, '%b-%Y') AS month_group,
4      ROUND(SUM(gross_revenue), 2) AS total_revenue,
5      ROUND(
6          LAG(SUM(gross_revenue), 1, SUM(gross_revenue)) OVER (
7              PARTITION BY brand_name
8              ORDER BY DATE_FORMAT(orders_date, '%Y-%m')
9          ),
10         2
11     ) AS previous_month,
12     ROUND(
13         SUM(gross_revenue) - LAG(SUM(gross_revenue), 1, SUM(gross_revenue)) OVER (
14             PARTITION BY brand_name
15             ORDER BY DATE_FORMAT(orders_date, '%Y-%m')
16         ),
17         2
18     ) AS MOM_growth
19 FROM fact_orders o
20 JOIN dim_brand b ON o.brand_id = b.brand_id
21 WHERE order_status = 'Delivered'
22 GROUP BY brand_name, DATE_FORMAT(orders_date, '%Y-%m'), DATE_FORMAT(orders_date, '%b-%Y')
23 ORDER BY brand_name, DATE_FORMAT(orders_date, '%Y-%m');
```

brand_name	month_group	total_revenue	previous_month	MOM_growth
Biryani Brand	Jan-2023	138054.67	138054.67	0
Biryani Brand	Feb-2023	113216.1	138054.67	-24838.57
Biryani Brand	Mar-2023	149480.97	113216.1	36264.87
Biryani Brand	Apr-2023	163084.76	149480.97	13603.79
Biryani Brand	May-2023	159080.81	163084.76	-4003.95
Biryani Brand	Jun-2023	186997.89	159080.81	27917.08
Biryani Brand	Jul-2023	210606.18	186997.89	23608.29
Biryani Brand	Aug-2023	218651.4	210606.18	8045.22
Biryani Brand	Sep-2023	208141.39	218651.4	-10510.01
Biryani Brand	Oct-2023	181159.84	208141.39	-26981.55
Biryani Brand	Nov-2023	218354.69	181159.84	37194.85
Biryani Brand	Dec-2023	222534.46	218354.69	4179.77
Biryani Brand	Jan-2024	230259.27	222534.46	7724.81
Biryani Brand	Feb-2024	198824.1	230259.27	-31435.17
Biryani Brand	Mar-2024	236270.65	198824.1	37446.55
Biryani Brand	Apr-2024	218543.34	236270.65	-17727.31
Biryani Brand	May-2024	223048.94	218543.34	4505.6
Biryani Brand	Jun-2024	276579.01	223048.94	53530.07
Burger Brand	Jan-2023	80028.24	80028.24	0
Burger Brand	Feb-2023	67493.55	80028.24	-12534.69
Burger Brand	Mar-2023	84004.09	67493.55	16510.54
Burger Brand	Apr-2023	84089.34	84004.09	85.25
Burger Brand	May-2023	80054.53	84089.34	-4034.81
Burger Brand	Jun-2023	94792.95	80054.53	14738.42
Burger Brand	Jul-2023	100744.89	94792.95	5951.94

Month-on-Month (MoM) Growth

It helps track revenue trends over time, making it easy to identify periods of business growth, decline, or stable performance for each brand.

```
1 CREATE VIEW vw_brand_unit_economics AS
2 SELECT
3     o.order_id,
4     b.brand_id,
5     b.brand_name,
6     t.time_slot_id,
7     t.slot_name,
8     t.day_type,
9     d.zone_id,
10    d.zone_name,
11    d.distance_km_band,
12    d.avg_delivery_time_min,
13    p.platform_id,
14    p.platform_name,
15    p.commission_rate,
16    o.orders_date,
17    o.gross_revenue,
18    o.discount_applied,
19    ROUND(o.gross_revenue - o.discount_applied,2) AS net_revenue,
20    ROUND((o.gross_revenue - o.discount_applied) * o.platform_commission_pct,2) AS platform_commission,
21    ROUND(o.food_cost + o.packing_cost + o.delivery_cost_brand,2) AS variable_cost,
22    ROUND((o.gross_revenue - o.discount_applied)
23        - ((o.gross_revenue - o.discount_applied) * o.platform_commission_pct)
24        - (o.food_cost + o.packing_cost + o.delivery_cost_brand),2) AS contribution_margin,
25    ROUND(((o.gross_revenue - o.discount_applied)
26        - ((o.gross_revenue - o.discount_applied) * o.platform_commission_pct)
27        - (o.food_cost + o.packing_cost + o.delivery_cost_brand)) * 100.0 / NULLIF((o.gross_revenue - o.discount_applied),0),2)
28        AS contribution_margin_pct
29 FROM fact_orders o
30 JOIN dim_brand b ON o.brand_id = b.brand_id
31 JOIN dim_time_slot t ON o.time_slot_id = t.time_slot_id
32 JOIN dim_delivery_zone d ON o.zone_id = d.zone_id
33 JOIN dim_platform p ON o.platform_id = p.platform_id
34 WHERE o.order_status = 'Delivered';
```

Brand Unit Economics View

It provides a reusable analytical layer that simplifies reporting and enables faster analysis of profitability across brands, platforms, time slots, and delivery zones without rewriting complex calculations.